4-2 · Apache Iceberg 레이크하우스 구축 · 이론 1H

레이크하우스와 테이블 포맷

파일 무더기를 테이블처럼 — 메타데이터 한 겹의 힘

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 파일만 쌓았을 때 무엇이 안 되는지 세 가지 이상 말할 수 있다

② 카탈로그 → 스냅샷 → 매니페스트 → 데이터 파일 사슬을 그린다

③ 커밋이 「카탈로그 포인터를 바꾸는 한 순간」임을 설명한다

④ 스키마를 바꿔도 데이터 파일을 다시 쓰지 않는 이유를 안다

다음 실습에서 이 사슬이 MinIO 안에 실제 파일로 생기는 것을 확인합니다

파일만 쌓으면 안 되는 것 여섯 가지

상황	파일만 쌓았을 때	왜 문제인가
쓰는 중에 읽음	반쯤 쓰인 파일이 목록에 보인다	집계가 틀린 값을 낸다
일부 행 수정	파일 전체를 다시 써야 한다	4-1 의 「부분 수정 불가」
파일 삭제	읽던 쿼리가 없는 파일을 찾는다	쿼리가 실패한다
컬럼 추가	옛 파일과 새 파일 모양이 다르다	읽는 쪽이 깨진다
어제 상태 보기	방법이 없다	잘못 덮어쓰면 복구 불가
동시에 쓰기	서로 덮어쓴다	데이터가 사라진다

오브젝트 스토리지는 파일 목록만 줄 뿐, 그중 무엇이 유효한지는 모른다 — 「지금 이 테이블의 최신 상태」를 아무도 보장하지 않는다

1. 파일만 쌓으면

데이터레이크와 레이크하우스 — 한 겹 차이

데이터레이크 — 파일만

오브젝트 스토리지 위에 파일을 쌓기만 한 상태

- 파일 목록만 있고 무엇이 유효한지 모른다
- 반쯤 쓰인 파일이 그대로 보인다
- 어제 상태로 돌아갈 방법이 없다

VS

레이크하우스 — 메타데이터 층을 얹음

데이터레이크에 DB 의 관리 능력을 더한 구조

- 트랜잭션 · 스키마 관리 · 시점 조회
- 테이블 포맷이 그 층을 맡는다
- Iceberg · Delta Lake · Hudi 가 대표

Iceberg 의 네 층 — 카탈로그에서 데이터 파일까지

카탈로그	스냅샷	매니페스트 목록	매니페스트	데이터 파일
테이블 이름 → 최신 메타데이터 위치	커밋 시점마다 테이블 전체 상태	이 스냅샷을 이루는 매니페스트들	데이터 파일 목록 + 행 수·min/max	실제 값이 든 parquet

스냅샷은 사진첩 — 수정할 때마다 새 사진을 한 장 찍어 두니 「지난달 사진 속 모습」을 언제든 다시 본다

각 층이 하는 일이 다르다 — 카탈로그 포인터 교체 = 커밋 · 매니페스트 min/max = 안 읽어도 될 파일 건너뛰기 · 옛 스냅샷 = 타임 트래블

3. 원자적 커밋

커밋은 포인터를 바꾸는 한 순간 — 반쯤 쓰인 상태가 없다

1. **새 데이터 파일들을 쓴다**
   아직 아무에게도 안 보인다
2. **새 매니페스트를 쓴다**
   아직 안 보인다
3. **새 스냅샷 메타데이터를 쓴다**
   아직 안 보인다
4. **카탈로그의 포인터를 바꾼다**
   이 순간 전부 한꺼번에 보인다 — 이것이 커밋

중간에 실패하면

4단계가 없으니
없던 일

4. 스키마 진화

스키마를 바꿔도 데이터 파일을 다시 쓰지 않는다

작업	가능한가	데이터 파일을 다시 쓰나
컬럼 추가	가능	안 씀 — 옛 파일은 NULL 로 읽힘
컬럼 삭제	가능	안 씀 — 읽을 때 무시
컬럼 이름 변경	가능	안 씀 — ID 로 추적
컬럼 순서 변경	가능	안 씀
타입 확장 int → bigint	가능	안 씀
타입 축소 bigint → int	불가	데이터가 잘릴 수 있어 막음

컬럼을 이름이 아니라 내부 ID 로 추적하기 때문 — Hive 는 이름·위치로 찾아 순서를 바꾸면 값이 뒤섞이는 사고가 났다

5. 카탈로그

카탈로그의 종류 — 오늘은 REST

카탈로그	어디에 저장	언제 쓰나
REST	별도 서비스	오늘 실습 · 엔진 독립적
Hive Metastore	RDB (MySQL 등)	기존 Hive 자산이 있을 때
Hadoop / 파일 기반	스토리지 자체	단순하지만 동시성이 약함
AWS Glue	AWS 관리형	AWS 를 쓸 때
JDBC	RDB	가볍게 시작할 때

REST 는 Spark 와 Trino 가 같은 카탈로그를 보게 한다(4-5 상호운용 검증) — 오늘은 테이블 목록을 볼륨 안 파일에 보관

정리

이 1시간에서 꼭 가져갈 것

1. 파일만 쌓으면 「무엇이 현재 상태인가」를 아무도 보장하지 않는다
2. Iceberg 는 카탈로그 → 스냅샷 → 매니페스트 → 데이터 파일 사슬로 보장한다
3. 커밋은 카탈로그 포인터를 바꾸는 한 순간 — 반쯤 쓰인 상태가 없다
4. 스냅샷을 남겨 두므로 과거 시점을 그대로 본다 (타임 트래블)
5. 컬럼을 ID 로 추적해 스키마를 바꿔도 데이터 파일을 다시 쓰지 않는다

4-2 · 이론 1H 마무리

다음은 Spark + Iceberg 환경 구성 (1H)

4-1 의 Compose 를 통째로 바꿔 REST 카탈로그와 Spark 를 더하고, 방금 본 사슬이 돌아갈 자리를 만들어 보세요

- ✓ **docker-compose.yml 통째로 교체 · 컨테이너 넷**
- ✓ **curl 로 minio · REST 카탈로그 확인**
- ✓ **spark-defaults.conf 의 demo 카탈로그 설정**
- ✓ **SHOW CATALOGS · 카탈로그 멈춰 보기**

준비할 것

4-1 의 ~/lakehouse · 미리 받은 이미지 2개